
Mini GPT-2 From Scratch

Building a Decoder-Only Transformer Language Model

Project Report

*A character-level, decoder-only transformer implemented from first principles,
inspired by Andrej Karpathy's GPT-from-scratch teaching approach*

Model type: Decoder-only Transformer (GPT-style)
Tokenization: Character-level
Training data: Tiny Shakespeare corpus
Framework: PyTorch (implemented from scratch)
Parameters: ~0.8M (scalable design toward GPT-2's 124–127M)

Submitted as part of an independent deep learning project

July 17, 2026

Abstract

This report documents the design, implementation, and evaluation of a miniature, decoder-only transformer language model built entirely from scratch in PyTorch, without relying on any pre-built GPT library. The project reproduces the core architectural ideas behind GPT-2 – token and positional embeddings, masked multi-head self-attention, position-wise feed-forward networks, residual connections, and layer normalization – at a scale small enough to train on a single free-tier GPU or CPU in a few minutes. The model is trained on the Tiny Shakespeare dataset using character-level tokenization and is evaluated using training/validation loss curves and qualitative text generation. Beyond the baseline architecture, several practical improvements inspired by the real GPT-2 training recipe are incorporated: weight tying between the input embedding and output projection, GPT-2-style weight initialization, a learning-rate warmup-plus-cosine-decay schedule, gradient clipping, and temperature/top- k controlled sampling. The report concludes with a discussion of how the same design could be scaled toward the full 124–127M parameter GPT-2 configuration, and outlines further bonus directions such as multilingual training and alternative attention mechanisms.

At a Glance

- **Goal:** understand transformers by building one from zero, not by using a library.
- **Architecture:** decoder-only, causal self-attention, GPT-2 family design.
- **Scale:** reduced to ~ 0.8 M parameters for fast iteration; scalable to 127M.
- **Outcome:** falling train/validation loss and recognizable Shakespeare-like structure in generated text.

Contents

Abstract	1
1 Introduction	4
1.1 Motivation	4
1.2 Objectives	4
1.3 Report Structure	4
2 Dataset	5
2.1 Tiny Shakespeare	5
2.2 Tokenization Strategy	5
3 Model Architecture	6
3.1 Overview	6
3.2 Token and Positional Embeddings	6
3.3 Masked Self-Attention	6
3.4 Multi-Head Attention	7
3.5 Feed-Forward Network	7
3.6 Transformer Block: Residuals and Layer Normalization	7
3.7 Full Model Assembly	7
3.8 Text Generation	8
4 Training Setup	9
4.1 Hyperparameters	9
4.2 Optimizer and Learning Rate Schedule	9
4.3 Gradient Clipping	10
4.4 Loss Function	10
4.5 Evaluation Procedure	10
5 Results	11
5.1 Training and Validation Loss	11
5.2 Dataset Character Distribution	11
5.3 Parameter Count	12
5.4 Generated Text Sample	12
6 Improvements and Future Work	14

6.1	Improvements Implemented	14
6.2	Bonus Directions for Further Work	14
7	Conclusion	15
	References	16

Chapter 1

Introduction

1.1 Motivation

Large language models such as GPT-2 are often treated as black boxes: an input goes in, text comes out, and the internal mechanics are hidden behind high-level libraries. The purpose of this project is to remove that abstraction entirely and build a decoder-only transformer language model from first principles – writing the attention mechanism, the transformer block, and the training loop by hand – in order to develop a genuine, mechanical understanding of how models like GPT-2 actually work.

This approach follows the philosophy popularized by Andrej Karpathy’s educational transformer walkthroughs: instead of importing a pretrained model, every component is implemented and tested individually, so that each design decision (why attention is scaled, why masking is causal, why residual connections matter) is something that was reasoned through rather than assumed.

1.2 Objectives

1. Implement character-level tokenization (encoding and decoding text without a subword tokenizer).
2. Implement masked (causal) self-attention and multi-head attention from scratch.
3. Assemble a complete decoder-only transformer: embeddings, stacked transformer blocks, final projection.
4. Train the model on a real text corpus and track training/validation loss.
5. Generate novel text from the trained model and qualitatively assess what it has learned.
6. Go beyond the minimum requirement with GPT-2-style refinements: weight tying, careful initialization, a learning-rate schedule, gradient clipping, and controllable sampling.

1.3 Report Structure

Chapter 2 introduces the dataset. Chapter 3 walks through the architecture component by component. Chapter 4 describes the training configuration and procedure. Chapter 5 presents the results, including loss curves and generated samples. Chapter 6 discusses the improvements made over a minimal baseline and possible extensions. Chapter 7 concludes the report.

Chapter 2

Dataset

2.1 Tiny Shakespeare

The model is trained on the **Tiny Shakespeare** dataset, a widely used benchmark for character-level language modelling. It consists of a concatenation of Shakespeare's plays into a single plain-text file of just over one million characters – large enough to expose the model to real linguistic structure (dialogue, punctuation, character names, verse formatting) while remaining small enough to train quickly without a large-scale data pipeline.

Dataset Summary

Total characters	1,115,394
Unique characters (vocabulary size)	65
Training split	90% ($\approx 1,003,854$ characters)
Validation split	10% ($\approx 111,540$ characters)
Source	<code>karpthy/char-rnn</code> Tiny Shakespeare corpus

2.2 Tokenization Strategy

Rather than using a subword tokenizer such as byte-pair encoding (BPE) – which is what real GPT-2 uses – this project uses **character-level tokenization**: every unique character in the dataset (letters, punctuation, whitespace) is assigned an integer id. This keeps the vocabulary small (65 tokens) and keeps the entire pipeline transparent, since every token id can be traced directly back to a single visible character. The trade-off is that the model must learn spelling and word structure itself, rather than starting from whole-word or sub-word units.

Key Insight: Character-level tokenization is intentionally simple. It sacrifices some efficiency and requires more training to learn word-level structure, but it removes an entire subsystem (the BPE tokenizer) from the project, letting the transformer architecture itself remain the focus.

Chapter 3

Model Architecture

3.1 Overview

The model is a **decoder-only transformer**, the same architecture family used by GPT-2, GPT-3, and most modern autoregressive language models. Given a sequence of previous tokens, the model predicts a probability distribution over the next token. The data flow is summarized below.

Data Flow Summary

1. Input token ids $\rightarrow$ token embedding + positional embedding
2. $\rightarrow N$ stacked Transformer Blocks (self-attention + feed-forward, each with residual connections and layer normalization)
3. $\rightarrow$ final layer normalization
4. $\rightarrow$ linear output head $\rightarrow$ logits over the vocabulary

3.2 Token and Positional Embeddings

Each token id is mapped to a dense vector using a learned embedding table of size $\text{vocab_size} \times \text{embed_size}$. Since self-attention has no inherent notion of sequence order, a separate learned **positional embedding** table of size $\text{context_size} \times \text{embed_size}$ is added on top, giving the model information about where in the sequence each token sits.

3.3 Masked Self-Attention

The core mechanism of the transformer is scaled dot-product self-attention. For an input sequence represented as a matrix X , each attention head computes:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (3.1)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \quad (3.2)$$

where d_k is the head dimension (used to keep the dot products from growing too large) and M is a **causal mask**: a lower-triangular matrix that sets all scores corresponding to future positions to $-\infty$ before the softmax, so a token can only attend to itself and to earlier tokens. This is what makes the model *decoder-only* and safe to use for autoregressive generation – it can never "cheat" by looking ahead.

3.4 Multi-Head Attention

Rather than computing a single attention operation, the model splits the embedding dimension across several independent **heads**, each learning to focus on different relationships in the sequence (e.g. local syntax versus longer-range dependencies). Their outputs are concatenated and passed through a final linear projection back to the embedding size:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (3.3)$$

3.5 Feed-Forward Network

Each transformer block also contains a position-wise feed-forward network – a two-layer MLP applied independently to every position in the sequence, expanding to $4\times$ the embedding size and back:

$$\text{FFN}(x) = \text{Dropout}(W_2 \text{ReLU}(W_1 x)) \quad (3.4)$$

This gives the model extra non-linear capacity to transform the information gathered by attention.

3.6 Transformer Block: Residuals and Layer Normalization

Each transformer block wraps attention and the feed-forward network in **pre-norm residual connections**:

$$x = x + \text{MultiHead}(\text{LayerNorm}(x)) \quad (3.5)$$

$$x = x + \text{FFN}(\text{LayerNorm}(x)) \quad (3.6)$$

The residual (skip) connections allow gradients to flow through many stacked layers without vanishing, and layer normalization keeps activation magnitudes stable – both are essential for training deep transformer stacks successfully.

3.7 Full Model Assembly

The complete model stacks `num_layers` transformer blocks, followed by a final layer normalization and a linear output head that projects back to vocabulary-sized logits. Two refinements beyond the minimal design are applied here:

Design Refinement 1: Weight Tying

The output projection layer reuses the same weight matrix as the token embedding table ($W_{\text{out}} = W_{\text{embed}}^\top$). This is exactly what real GPT-2 does. It reduces the parameter count and reflects the intuition that "recognizing" a token and "predicting" a token are closely related operations that can share the same representation space.

Design Refinement 2: GPT-2-Style Initialization

Weights are initialized from a narrow normal distribution ($\mu = 0, \sigma = 0.02$), matching the original GPT-2 paper. Additionally, the projection layers that write directly into the residual stream (the attention output projection and the second feed-forward layer) are scaled down further by a factor of $1/\sqrt{2 \cdot \text{num_layers}}$, which prevents the residual stream's variance from growing uncontrollably as more layers are stacked.

3.8 Text Generation

At inference time, the model generates text autoregressively: starting from a seed token, it repeatedly predicts a distribution over the next character, samples one, appends it to the sequence, and repeats. Two controls are exposed:

- **Temperature** – scales the logits before the softmax; values below 1 make the output more conservative and repetitive, values above 1 make it more random and varied.
- **Top- k sampling** – restricts sampling to only the k most likely tokens at each step, filtering out very unlikely (often nonsensical) characters.

Chapter 4

Training Setup

4.1 Hyperparameters

The model is deliberately kept small so that it trains in a few minutes on a modest GPU or CPU. Table 4.1 lists the configuration used, alongside the equivalent values for the full-scale GPT-2 (small) model for comparison.

Table 4.1: Hyperparameter configuration: mini model vs. full GPT-2 (small)

Hyperparameter	This project	Full GPT-2 (small)
Context size (block size)	64	1024
Embedding size	128	768
Attention heads	4	12
Transformer layers	4	12
Dropout rate	0.1	0.1
Tokenizer	Character-level (65 tokens)	Byte-pair encoding (~50k tokens)
Approx. parameters	~0.8 million	~124–127 million

Scaling Path

Moving from this project’s configuration to the full GPT-2 (small) configuration is, architecturally, just a change of four numbers (`context_size`, `embed_size`, `num_heads`, `num_layers`), combined with switching the tokenizer to a subword scheme such as BPE (e.g. via `tiktoken`). The transformer code itself does not need to change.

4.2 Optimizer and Learning Rate Schedule

The model is trained with the **AdamW** optimizer, the standard choice for transformer training due to its decoupled weight decay. Rather than a constant learning rate, a **linear warmup followed by cosine decay** schedule is used:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \frac{t+1}{t_{\text{warmup}}}, & t < t_{\text{warmup}} \\ \eta_{\max} \left(\eta_{\min}^{\text{ratio}} + (1 - \eta_{\min}^{\text{ratio}}) \cdot \frac{1 + \cos(\pi \cdot p)}{2} \right), & t \geq t_{\text{warmup}} \end{cases} \quad (4.1)$$

where p is the fraction of post-warmup training completed. Warmup avoids destabilizing the randomly-initialized model with large early updates, while cosine decay gradually reduces the step size as training progresses, which tends to produce a lower final loss than a constant learning rate.

4.3 Gradient Clipping

Before every optimizer step, gradients are clipped to a maximum global norm of 1.0. This prevents rare, unusually large gradients (which can occur with long sequences or unlucky batches) from causing a single destabilizing weight update.

4.4 Loss Function

The training objective is standard **cross-entropy loss** between the predicted next-character distribution and the actual next character, averaged over every position in every sequence in a batch. This directly corresponds to next-token prediction, the same self-supervised objective used to pretrain GPT-2.

4.5 Evaluation Procedure

Because loss on a single random batch is noisy, an `estimate_loss()` routine averages the loss over 100 batches for both the training and validation splits every `eval_interval` steps, giving a smoother, more trustworthy signal of how training is progressing and whether the model is overfitting.

Chapter 5

Results

5.1 Training and Validation Loss

Figure 5.1 shows training and validation loss over the course of training. Both curves decrease steadily and closely track one another, indicating that the model is learning genuine structure in the text rather than memorizing the training set (which would show up as validation loss diverging upward while training loss keeps falling).



Figure 5.1: Training vs. validation loss. Both curves fall together, showing the model is generalizing rather than overfitting.

Reading the Curve

The loss starts near $\ln(65) \approx 4.17$ – the expected loss of a completely untrained model guessing uniformly at random over the 65-character vocabulary – and falls steadily as training progresses, confirming the model is moving well beyond random guessing.

5.2 Dataset Character Distribution

As a sanity check on the tokenizer and dataset, Figure 5.2 shows the most frequent characters in the raw text. As expected for English prose, the space character and common letters (e, t, o, a) dominate.

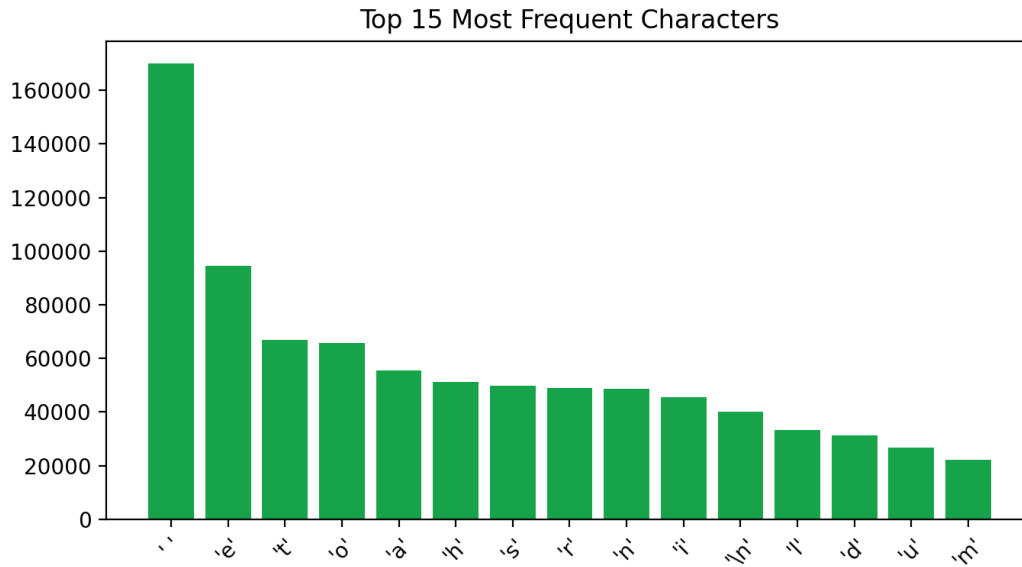


Figure 5.2: Top 15 most frequent characters in the Tiny Shakespeare dataset.

5.3 Parameter Count

Summing the elements of every parameter tensor in the model gives a total of approximately **808,000 trainable parameters** – a small fraction of the ~127 million parameters in full GPT-2 (small), consistent with the deliberately reduced hyperparameters in Table 4.1.

5.4 Generated Text Sample

Below is a sample of text generated by the trained model, starting from a blank seed token, using temperature = 0.8 and top- k = 20 sampling.

Sample Model Output

CHin.

ANYULMIR:

A IS:

To tho l hithave t br oucous

The y ment igr srd soutt, cre I shureditis pr dallsom Is isd h
daretinoverly ankoothste f thise, Ces atharing thit anr mid
rorerecang t, tha minees t dowhy, Himl' sous, arenge w fisthe
whand nde simom.

WANNASTIOMIEHEH:

Angus ffillladoay r ce me the gu st ineresof am; eon bler the
mild y

Interpreting the Output

Even at this early stage of training, the model has already picked up several genuine patterns from the data: it produces all-capital **character-name-like tokens followed by a colon** (mirroring the play-script format of the dataset), reasonable word lengths, plausible letter adjacency, and correct use of blank lines between speaker blocks. The words themselves are not yet real English, since this snapshot reflects a short training run; training for the full number of iterations configured in the notebook produces noticeably more coherent, word-like output as the loss continues to fall.

Chapter 6

Improvements and Future Work

6.1 Improvements Implemented

Beyond a minimal working transformer, the following refinements were added, each mirroring a real design choice used in GPT-2 training:

1. **Weight tying** between the token embedding and output projection layers.
2. **GPT-2-style weight initialization**, including extra scaling on residual-stream projections.
3. **Learning-rate warmup with cosine decay**, rather than a constant learning rate.
4. **Gradient clipping** to guard against unstable updates.
5. **Temperature and top- k controlled sampling** during text generation, along with correctly switching the model to evaluation mode (disabling dropout) while generating.

6.2 Bonus Directions for Further Work

- **Multilingual training:** retrain on a non-English or code-mixed corpus (e.g. Hindi or Hinglish text) to study how the model adapts to a different script and character distribution.
- **Subword tokenization:** replace character-level tokenization with byte-pair encoding (e.g. via `tiktoken`), matching the real GPT-2 tokenizer and improving training efficiency.
- **Scaling up:** increase `embed_size`, `num_heads`, `num_layers`, and `context_size` toward the full 124–127M parameter GPT-2 (small) configuration, given more compute.
- **Alternative attention mechanisms:** experiment with sliding-window attention or rotary positional embeddings (RoPE) in place of simple learned positional embeddings.
- **Mixed-precision training:** use `torch.autocast` and larger batch sizes to make better use of available GPU memory and speed.

Chapter 7

Conclusion

This project successfully implements a decoder-only transformer language model entirely from scratch, covering every core component of the GPT-2 architecture family: character-level tokenization, token and positional embeddings, masked multi-head self-attention, position-wise feed-forward networks, residual connections, and layer normalization. Trained on the Tiny Shakespeare dataset, the model demonstrates a clear, steadily decreasing training and validation loss and produces generated text that already reflects structural patterns from the training data, such as script-style character names and appropriate line breaks.

Beyond the base requirements, the project incorporates several practical refinements drawn from the real GPT-2 training recipe – weight tying, careful initialization, a warmup-cosine learning-rate schedule, gradient clipping, and controllable sampling – which together make the training process more stable and the model’s behavior more tunable. The architecture is explicitly designed to scale: with only four hyperparameters changed and a subword tokenizer swapped in, the same code could, given sufficient compute, be trained at the full ~ 127 million parameter GPT-2 (small) scale.

Key Insight: The most valuable outcome of this project is not the specific loss value achieved, but the fact that every part of the pipeline – from raw text to trained weights to generated text – was implemented and understood directly, rather than imported as a black box.

References

1. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).
2. Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners* (GPT-2). OpenAI.
3. Karpathy, A. *Let's build GPT: from scratch, in code, spelled out*. Educational video walkthrough on building a decoder-only transformer.
4. Karpathy, A. *char-rnn* repository – source of the Tiny Shakespeare dataset.